

Tutorial 7 — Simulation Walkthrough

A Step-by-Step Code Guide to Type I/II Errors, Power, and Confidence Intervals

Introductory Econometrics — TA Session

Read this alongside the problem set. Every simulation built from scratch.

Contents

1	Two Essential R Functions: <code>pnorm</code> and <code>qnorm</code>	3
1.1	<code>pnorm</code> : the area to the left of a value	3
1.2	<code>qnorm</code> : the inverse — find the value that gives a target area	3
2	Simulating One Hypothesis Test from Scratch	5
2.1	Step 1 — Define the world	5
2.2	Step 2 — Observe one estimate	5
2.3	Step 3 — Compute the test statistic	6
2.4	Step 4 — Make a decision	6
2.5	Step 5 — Label the outcome	6
3	Repeating 10,000 Times: Verifying the Size	8
3.1	Step 1 — Decide how many repetitions	8
3.2	Step 2 — Draw 10,000 estimates in one line	8
3.3	Step 3 — Compute all 10,000 test statistics	8
3.4	Step 4 — Apply the rejection rule to all experiments	9
3.5	Step 5 — Visualise the distribution of Z statistics	9
4	The Power Function: What Happens When H_0 is False?	11
4.1	Step 1 — What changes when H_0 is false?	11
4.2	Step 2 — Compute power for $\beta^* = 6$ analytically	11
4.3	Step 3 — Verify with simulation	12
4.4	Step 4 — Compute power for every possible true value	12
4.5	Step 5 — How does sample size move the curve?	13
5	The OLS Estimator: Building Sampling Intuition	15
5.1	Step 1 — Define the data-generating process (DGP)	15
5.2	Step 2 — Draw one sample and inspect it	15
5.3	Step 3 — Fit OLS and read off the estimate	16
5.4	Step 4 — Repeat 40 times: the spaghetti plot	17
5.5	Step 5 — Repeat 2,000 times: the sampling distribution	19
6	Confidence Intervals: What Does 95% Actually Mean?	21
6.1	Step 1 — How does <code>confint()</code> build the interval?	21
6.2	Step 2 — Check if this specific CI captures the true value	22
6.3	Step 3 — Repeat for 100 independent samples	22
6.4	Step 4 — Visualise all 100 intervals	23
6.5	Step 5 — Confirm coverage across different confidence levels	25

7 Quick Reference: Every Function Used**27**

Two Essential R Functions: `pnorm` and `qnorm`

Before writing any simulation we need to be fluent with two functions. They are the backbone of everything that follows.

1.1 `pnorm`: the area to the left of a value

`pnorm(x, mean, sd)` answers: “what fraction of a normal distribution lies below the value x ?”

```
# The standard normal N(0,1). How much area lies below 1.96?
pnorm(1.96, mean = 0, sd = 1)
```

```
## [1] 0.9750021
```

```
# By symmetry, the area above 1.96 is:
1 - pnorm(1.96, mean = 0, sd = 1)
```

```
## [1] 0.0249979
```

```
# The area in BOTH tails beyond +/- 1.96 (= alpha for a two-sided test):
(1 - pnorm(1.96)) + pnorm(-1.96)
```

```
## [1] 0.04999579
```

That last line gives us exactly 0.05. This is *why* 1.96 is the magic number for $\alpha = 0.05$.

1.2 `qnorm`: the inverse — find the value that gives a target area

`qnorm(p, mean, sd)` answers: “what value x has exactly fraction p of the distribution below it?”

```
# What value cuts off 97.5% of N(0,1) to the left?
qnorm(0.975)    # answer: 1.96 -- the two-sided critical value at alpha = 0.05
```

```
## [1] 1.959964
```

```
# What value cuts off 95% (one-sided critical value)?
qnorm(0.95)     # answer: 1.645
```

```
## [1] 1.644854
```

```
# What value cuts off 99.5% (for alpha = 0.01, two-sided)?
qnorm(0.995)    # answer: 2.576
```

```
## [1] 2.575829
```

The relationship between the two functions

`qnorm` and `pnorm` are inverses of each other:

$$\text{pnorm}(\text{qnorm}(p)) = p \quad \text{and} \quad \text{qnorm}(\text{pnorm}(x)) = x$$

The critical value $z_{0.025} = 1.96$ is defined by the equation $\Phi(1.96) = 0.975$. In R: `qnorm(0.975) = 1.96`. We will use this constantly.

```
# Verify: pnorm(qnorm(0.975)) should return exactly 0.975
pnorm(qnorm(0.975))
```

```
## [1] 0.975
```

```
# Verify: qnorm(pnorm(1.96)) should return exactly 1.96  
qnorm(pnorm(1.96))
```

```
## [1] 1.96
```

Simulating One Hypothesis Test from Scratch

Goal: understand exactly what happens in a single hypothesis test before we repeat it thousands of times.

Setting: test $H_0: \beta = 0$ vs. $H_1: \beta \neq 0$ at $\alpha = 0.05$, with known $\sigma_{\hat{\beta}} = 2$.

2.1 Step 1 — Define the world

```
# -- The TRUE state of the world (unknown to the researcher) --
beta_true <- 0      # H0 is TRUE: the real parameter is zero
sigma_hat <- 2      # the known standard deviation of beta_hat

# -- The researcher's decision rule --
alpha <- 0.05
z_crit <- qnorm(1 - alpha / 2)  # = 1.96 for a two-sided test

cat("True beta:      ", beta_true, "\n")

## True beta:      0

cat("sigma_beta_hat: ", sigma_hat, "\n")

## sigma_beta_hat:  2

cat("Critical value: ", round(z_crit, 4), "\n")

## Critical value:  1.96

cat("We reject H0 when |Z| >", round(z_crit, 4), "\n")

## We reject H0 when |Z| > 1.96
```

2.2 Step 2 — Observe one estimate

The researcher collects data and computes $\hat{\beta}$. In our simulation, we represent this by drawing one value from $N(\beta_{\text{true}}, \sigma_{\hat{\beta}}^2)$.

```
set.seed(101)  # fix seed so this specific draw is reproducible

# Draw one OLS estimate from the sampling distribution of beta_hat.
# Under our DGP, beta_hat ~ N(beta_true, sigma_hat^2).
# rnorm(n, mean, sd) draws n values from a normal distribution.
beta_hat_observed <- rnorm(1, mean = beta_true, sd = sigma_hat)

cat("The observed estimate is: beta_hat =", round(beta_hat_observed, 4), "\n")

## The observed estimate is: beta_hat = -0.6521

cat("The true value is:      beta      =", beta_true, "\n")

## The true value is:      beta      = 0

cat("Difference (due to noise): ", round(beta_hat_observed - beta_true, 4), "\n")

## Difference (due to noise): -0.6521
```

2.3 Step 3 — Compute the test statistic

The test statistic standardises the estimate: it measures how many standard deviations $\hat{\beta}$ sits away from the null value $\beta_0 = 0$.

```
# Z = (beta_hat - beta_0) / sigma_hat
# Under H0 (beta_0 = 0): Z = beta_hat / sigma_hat
beta_0 <- 0      # the null hypothesis value

Z_observed <- (beta_hat_observed - beta_0) / sigma_hat

cat("Test statistic: Z = (", round(beta_hat_observed,3), "- 0) /", sigma_hat,
    "=", round(Z_observed, 4), "\n")

## Test statistic: Z = ( -0.652 - 0) / 2 = -0.326
cat("Critical value: +/-", round(z_crit, 4), "\n")

## Critical value: +/- 1.96
cat("|Z| =", round(abs(Z_observed), 4), "\n")

## |Z| = 0.326
```

2.4 Step 4 — Make a decision

```
# The rejection rule: reject H0 if |Z| > z_crit
reject_H0 <- abs(Z_observed) > z_crit

if (reject_H0) {
  cat("DECISION: Reject H0 (|Z| =", round(abs(Z_observed),3),
      "> z_crit =", round(z_crit,3), ")\n")
} else {
  cat("DECISION: Fail to reject H0 (|Z| =", round(abs(Z_observed),3),
      "<= z_crit =", round(z_crit,3), ")\n")
}

## DECISION: Fail to reject H0 (|Z| = 0.326 <= z_crit = 1.96 )
```

2.5 Step 5 — Label the outcome

Since we *know* the true $\beta = 0$ (we set it in Step 1), we can classify this result.

```
# Two possibilities:
# - If we rejected H0 and H0 is TRUE -> Type I Error
# - If we did not reject H0 and H0 is TRUE -> Correct decision

if (reject_H0 & beta_true == 0) {
  cat("OUTCOME: Type I Error (false positive).\n")
  cat("          We rejected H0 even though beta = 0 is TRUE.\n")
} else if (!reject_H0 & beta_true == 0) {
  cat("OUTCOME: Correct. We did not reject H0, and H0 is indeed TRUE.\n")
}

## OUTCOME: Correct. We did not reject H0, and H0 is indeed TRUE.
```

What just happened?

We simulated one experiment. The true $\beta = 0$, so H_0 is true. Whether we made a Type I error or not depends entirely on the random draw of $\hat{\beta}$. In any one experiment, we cannot tell if we are right or wrong — we never observe the truth. The 5% guarantee is about what happens *over many repetitions*, which is exactly what we simulate next.

Repeating 10,000 Times: Verifying the Size

Goal: repeat the exact procedure from Section 2 ten thousand times and count how often we (incorrectly) reject H_0 when it is true. The answer should be approximately 5%.

3.1 Step 1 — Decide how many repetitions

```
n_sim      <- 10000    # number of simulated experiments
beta_true  <- 0        # H0 is TRUE in all 10,000 experiments
sigma_hat  <- 2
z_crit     <- qnorm(0.975) # = 1.96

cat("We will run", n_sim, "experiments.\n")

## We will run 10000 experiments.
cat("In every experiment, the true beta =", beta_true, "(H0 is always TRUE).\n")

## In every experiment, the true beta = 0 (H0 is always TRUE).
cat("We reject whenever |Z| >", round(z_crit, 4), "\n")

## We reject whenever |Z| > 1.96
```

3.2 Step 2 — Draw 10,000 estimates in one line

Instead of using a loop, R lets us draw all 10,000 values at once with a single `rnorm()` call.

```
# rnorm(n_sim, ...) draws n_sim independent values simultaneously.
# Each value represents beta_hat from one hypothetical experiment.
beta_hat_all <- rnorm(n_sim, mean = beta_true, sd = sigma_hat)

# Inspect the first 10 estimates
cat("First 10 simulated estimates of beta_hat:\n")

## First 10 simulated estimates of beta_hat:
round(head(beta_hat_all, 10), 3)

## [1]  1.105 -1.350  0.429  0.622  2.348  1.238 -0.225  1.834 -0.447  1.053

# Verify: the average should be close to the true beta = 0 (unbiasedness)
cat("Mean of 10,000 estimates:", round(mean(beta_hat_all), 4),
    " (should be close to", beta_true, ")\n")

## Mean of 10,000 estimates: 0.0103 (should be close to 0 )

cat("SD of 10,000 estimates: ", round(sd(beta_hat_all), 4),
    " (should be close to sigma_hat =", sigma_hat, ")\n")

## SD of 10,000 estimates:  1.9866 (should be close to sigma_hat = 2 )
```

3.3 Step 3 — Compute all 10,000 test statistics

```
# Standardise each estimate: Z = beta_hat / sigma_hat
# This produces 10,000 Z-statistics, one per simulated experiment.
Z_all <- beta_hat_all / sigma_hat
```



```
cat("First 10 test statistics:\n")

## First 10 test statistics:
round(head(Z_all, 10), 3)

## [1] 0.552 -0.675 0.214 0.311 1.174 0.619 -0.113 0.917 -0.223 0.526
# Under H0, each Z should look like a draw from N(0,1).
# Check: mean should be ~0, SD should be ~1.
cat("\nMean of Z statistics:", round(mean(Z_all), 4), " (expected: 0)\n")

##
## Mean of Z statistics: 0.0052 (expected: 0)
cat("SD of Z statistics: ", round(sd(Z_all), 4), " (expected: 1)\n")

## SD of Z statistics: 0.9933 (expected: 1)
```

3.4 Step 4 — Apply the rejection rule to all experiments

```
# This produces a logical (TRUE/FALSE) vector of length 10,000.
# TRUE = we rejected H0 in that experiment.
# FALSE = we did not reject H0 in that experiment.
rejected_all <- abs(Z_all) > z_crit

# How many rejections in total?
cat("Total rejections (out of", n_sim, "):", sum(rejected_all), "\n")

## Total rejections (out of 10000 ): 477
# The proportion of rejections = empirical Type I error rate.
# mean() on a TRUE/FALSE vector returns the fraction of TRUEs.
empirical_size <- mean(rejected_all)
cat("Empirical Type I error rate:", round(empirical_size, 4),
    " -- target: alpha =", alpha, "\n")

## Empirical Type I error rate: 0.0477 -- target: alpha = 0.05
```

3.5 Step 5 — Visualise the distribution of Z statistics

```
library(ggplot2)

df_size <- data.frame(Z = Z_all, rejected = rejected_all)

ggplot(df_size, aes(x = Z, fill = rejected)) +
  geom_histogram(bins = 80, color = "white", linewidth = 0.1) +
  scale_fill_manual(
    values = c("TRUE" = "#cc2222", "FALSE" = "#aabbdd"),
    labels = c("TRUE" = paste0("Rejected H0 (Type I error, n = ", sum(rejected_all), ")"),
               "FALSE" = "Correctly not rejected"),
    name = NULL) +
  geom_vline(xintercept = c(-z_crit, z_crit), linetype = "dashed", linewidth = 0.8) +
  annotate("text", x = z_crit + 0.15, y = 350, label = "+1.96", hjust = 0, size = 3) +
  annotate("text", x = -z_crit - 0.15, y = 350, label = "-1.96", hjust = 1, size = 3) +
  labs(
```

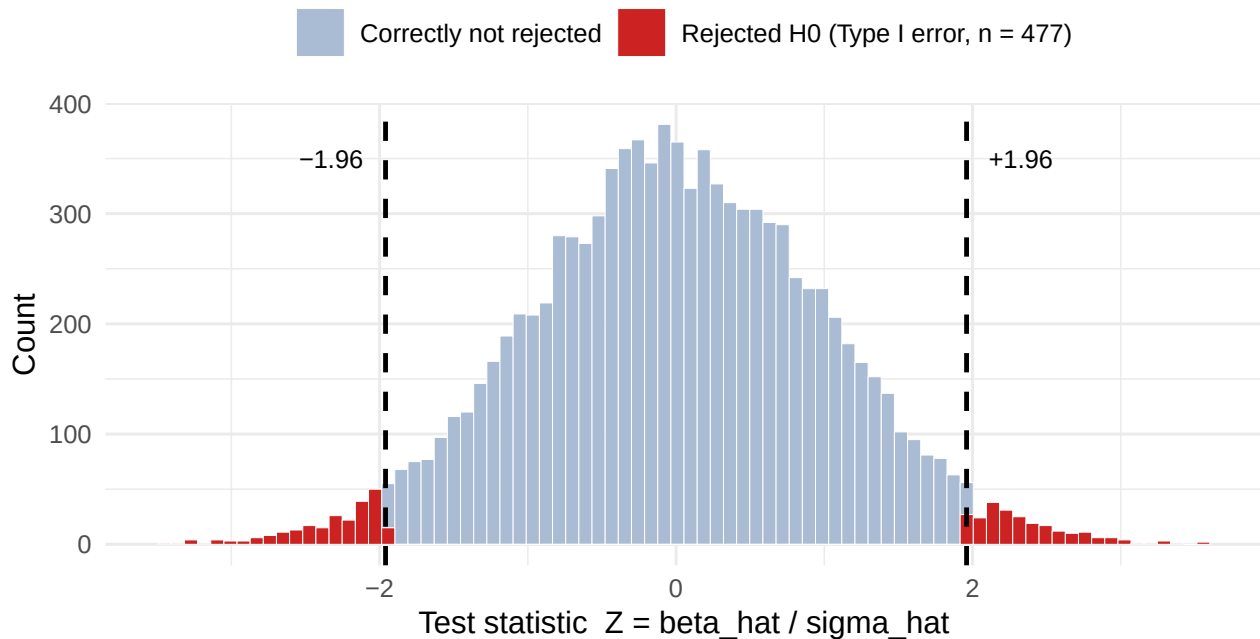
```

title    = paste0("Distribution of Z across ", n_sim, " experiments (H0 true)",
subtitle = paste0("Red = Type I errors (", round(empirical_size*100,1),
                  "% of all experiments) -- very close to alpha = 5%"),
x        = "Test statistic Z = beta_hat / sigma_hat",
y        = "Count"
) +
theme_minimal(base_size = 11) +
theme(legend.position = "top", plot.title = element_text(face = "bold"))

```

Distribution of Z across 10000 experiments (H0 true)

Red = Type I errors (4.8% of all experiments) -- very close to $\alpha = 5\%$



What this histogram tells us

Under H_0 , the test statistics follow $N(0, 1)$ — the histogram should look like a bell curve centred at zero. The dashed lines at ± 1.96 define the rejection region. The red bars (roughly 5% of all bars) are the Type I errors: experiments where $|Z| > 1.96$ purely by chance, even though the truth is $\beta = 0$.

The critical value 1.96 was engineered so that exactly 5% of $N(0, 1)$ falls in the tails. This simulation confirms it empirically.

The Power Function: What Happens When H_0 is False?

Goal: understand how the probability of *correctly* rejecting H_0 (= power) depends on the true effect size β^* .

4.1 Step 1 — What changes when H_0 is false?

When the true $\beta = \beta^*$ (not zero), the estimate $\hat{\beta}$ is no longer centred at zero — it is centred at β^* . The test statistic therefore follows $N(\delta, 1)$ instead of $N(0, 1)$, where $\delta = \beta^* / \sigma_{\hat{\beta}}$ is the signal-to-noise ratio.

```
sigma_hat <- 2
z_crit    <- qnorm(0.975)

# When beta* = 6: the test statistic Z = beta_hat / 2 ~ N(6/2, 1) = N(3, 1)
beta_star <- 6
delta     <- beta_star / sigma_hat # signal-to-noise ratio

cat("True effect:          beta* =", beta_star, "\n")

## True effect:          beta* = 6
cat("Signal-to-noise:     delta = beta*/sigma_hat =", delta, "\n")

## Signal-to-noise:     delta = beta*/sigma_hat = 3
cat("Distribution of Z:    N(delta, 1) = N(", delta, ", 1)\n")

## Distribution of Z:    N(delta, 1) = N( 3 , 1)
cat("Distribution under H0: N(0, 1)\n")

## Distribution under H0: N(0, 1)
cat("The distribution has shifted RIGHT by", delta, "units.\n")

## The distribution has shifted RIGHT by 3 units.
```

4.2 Step 2 — Compute power for $\beta^* = 6$ analytically

Power = $P(|Z| > 1.96)$ when $Z \sim N(\delta, 1)$. We split into two tails.

```
# RIGHT tail: P(Z > 1.96) when Z ~ N(delta, 1)
# Standardise: let W = Z - delta ~ N(0,1), then Z > 1.96 <=> W > 1.96 - delta
right_tail <- 1 - pnorm(z_crit - delta) # = P(W > z_crit - delta)

# LEFT tail: P(Z < -1.96) when Z ~ N(delta, 1)
# Z < -1.96 <=> W < -1.96 - delta
left_tail  <- pnorm(-z_crit - delta) # = P(W < -z_crit - delta)

power_beta6 <- right_tail + left_tail

cat("Right tail contribution: P(Z > 1.96) =", round(right_tail, 4), "\n")

## Right tail contribution: P(Z > 1.96) = 0.8508
cat("Left tail contribution: P(Z < -1.96) =", round(left_tail, 4), "\n")

## Left tail contribution: P(Z < -1.96) = 0
```

```
cat("Power at beta* = 6:      ", round(power_beta6, 4),
    "-- i.e.", round(power_beta6*100, 1), "%\n")
```

```
## Power at beta* = 6:      0.8508 -- i.e. 85.1 %
```

4.3 Step 3 — Verify with simulation

```
set.seed(42)
n_sim_power <- 10000

# Simulate 10,000 experiments where the TRUE beta = 6 (H0 is FALSE)
beta_hat_h1 <- rnorm(n_sim_power, mean = beta_star, sd = sigma_hat)
Z_h1 <- beta_hat_h1 / sigma_hat # test statistic (computed as if H0 were true)
rejected_h1 <- abs(Z_h1) > z_crit # apply the decision rule

simulated_power <- mean(rejected_h1) # fraction of correct rejections = power

cat("Analytical power at beta* =", beta_star, ":", round(power_beta6, 4), "\n")

## Analytical power at beta* = 6 : 0.8508
cat("Simulated power at beta* =", beta_star, ":", round(simulated_power, 4), "\n")

## Simulated power at beta* = 6 : 0.8445
cat("They agree: the formula is correct.\n")
```

```
## They agree: the formula is correct.
```

4.4 Step 4 — Compute power for every possible true value

Now we repeat Steps 2–3 for a grid of values $\beta^* \in [-8, 8]$ to draw the full power curve.

```
# Grid of candidate true values
beta_grid <- seq(-8, 8, by = 0.1)

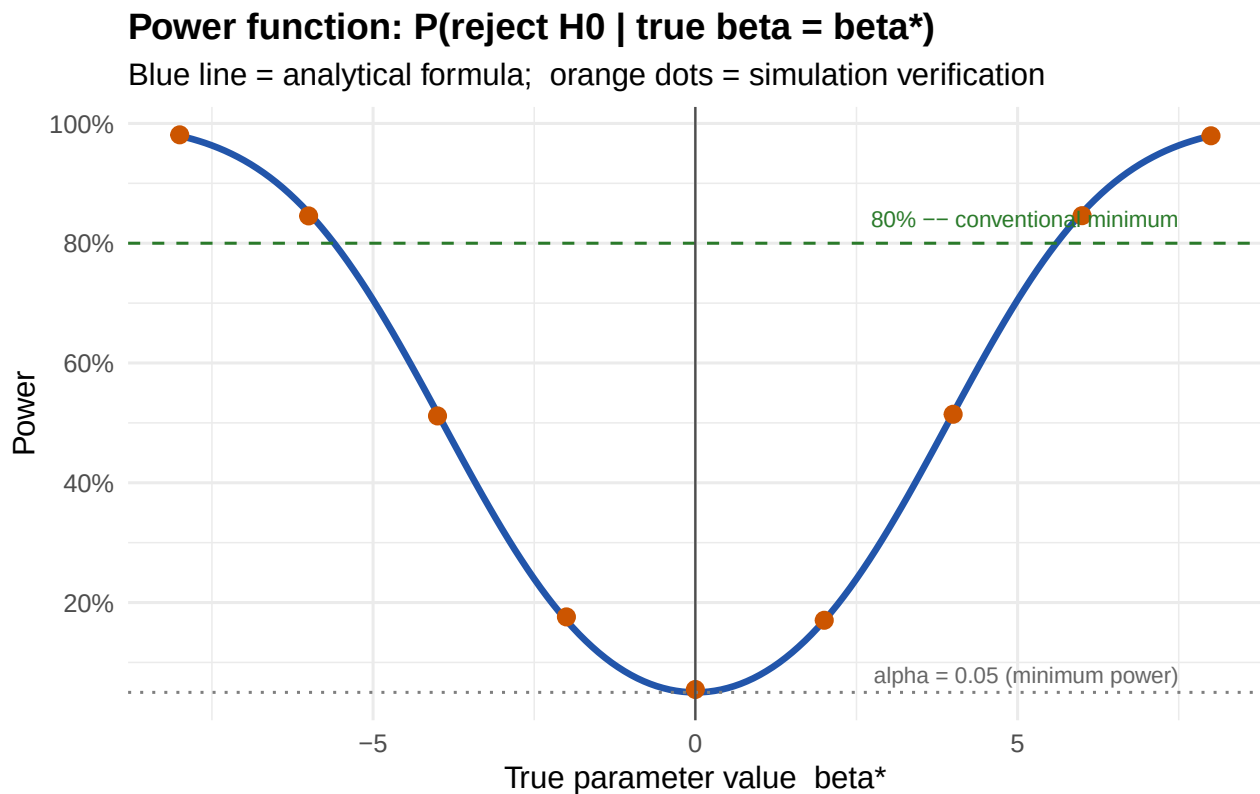
# For each value on the grid, compute delta and then power analytically
delta_grid <- beta_grid / sigma_hat
power_curve <- pnorm(-z_crit - delta_grid) + (1 - pnorm(z_crit - delta_grid))

# Simulation check at a coarser grid (5,000 draws per point)
beta_check <- seq(-8, 8, by = 2)
power_check <- sapply(beta_check, function(b) {
  Z_check <- rnorm(5000, mean = b, sd = sigma_hat) / sigma_hat
  mean(abs(Z_check) > z_crit)
})

# Plot
df_curve <- data.frame(beta = beta_grid, power = power_curve)
df_check <- data.frame(beta = beta_check, power = power_check)

ggplot() +
  geom_line(data = df_curve, aes(x = beta, y = power),
            color = "#2255aa", linewidth = 1.1) +
  geom_point(data = df_check, aes(x = beta, y = power),
             color = "#cc5500", size = 2.5) +
```

```
geom_hline(yintercept = 0.05, linetype = "dotted", color = "grey50") +
geom_hline(yintercept = 0.80, linetype = "dashed", color = "#2a7a2a") +
geom_vline(xintercept = 0, color = "grey30", linewidth = 0.4) +
annotate("text", x = 7.5, y = 0.08, label = "alpha = 0.05 (minimum power)",
         size = 2.8, hjust = 1, color = "grey40") +
annotate("text", x = 7.5, y = 0.84, label = "80% -- conventional minimum",
         size = 2.8, hjust = 1, color = "#2a7a2a") +
scale_y_continuous(labels = scales::percent, breaks = seq(0, 1, 0.2)) +
labs(title = "Power function: P(reject H0 | true beta = beta*)",
     subtitle = "Blue line = analytical formula; orange dots = simulation verification",
     x = "True parameter value beta*", y = "Power") +
theme_minimal(base_size = 11) +
theme(plot.title = element_text(face = "bold"))
```



4.5 Step 5 — How does sample size move the curve?

```
# The SE formula from Wooldridge Theorem 2.2:
# SE(beta_hat) = sigma / sqrt(SST_x)
#
# For x ~ Uniform(0, 10):
# E[SST_x] = n * Var(x) = n * 10^2/12
# => SE = sigma_u / sqrt(n * 100/12)
#
# Larger n => smaller SE => larger delta = beta*/SE => higher power

sigma_u <- 4    # error standard deviation
x_range <- 10   # x ~ Uniform(0, x_range)
```

```

# For each sample size, recompute the power curve with the appropriate SE
n_values <- c(20, 50, 200)
power_by_n <- do.call(rbind, lapply(n_values, function(n) {
  se_n <- sigma_u / sqrt(n * x_range^2 / 12) # SE for this n
  delta_n <- beta_grid / se_n
  power_n <- pnorm(-z_crit - delta_n) + (1 - pnorm(z_crit - delta_n))

  cat("n =", sprintf("%3d", n),
      " SE(beta_hat) =", round(se_n, 3),
      " Power at beta*=2:", round(power_n[beta_grid == 2], 3), "\n")

  data.frame(beta = beta_grid, power = power_n, n = as.character(n))
}))

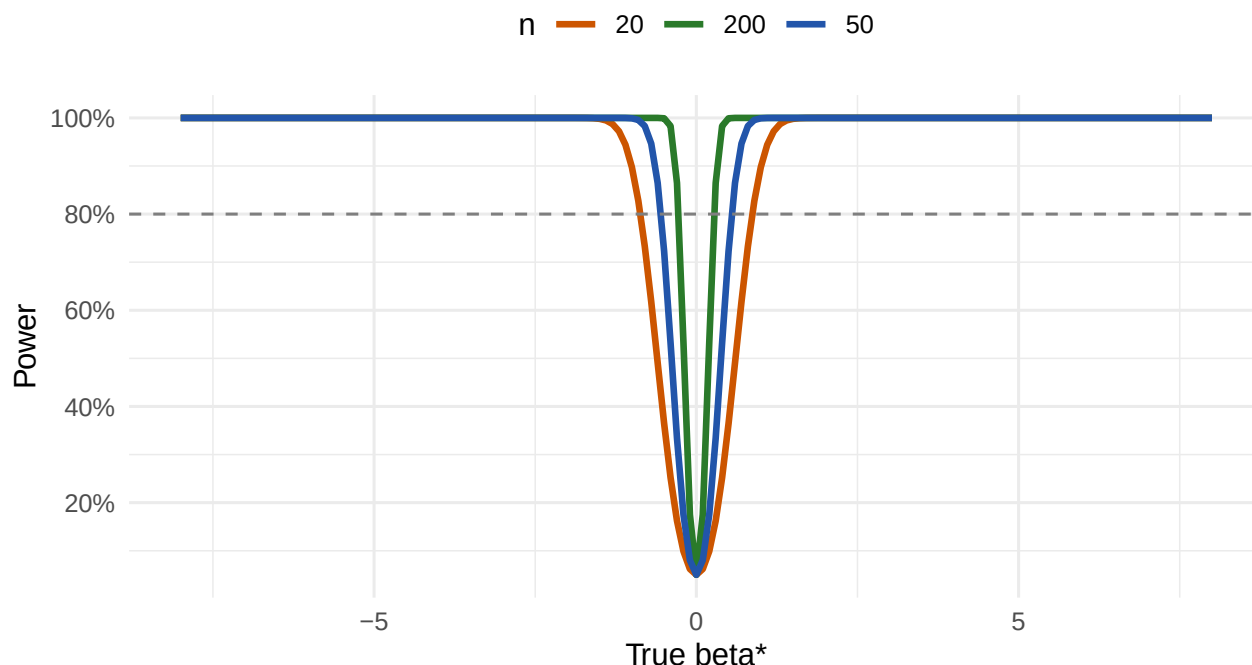
## n = 20 SE(beta_hat) = 0.31 Power at beta*=2: 1
## n = 50 SE(beta_hat) = 0.196 Power at beta*=2: 1
## n = 200 SE(beta_hat) = 0.098 Power at beta*=2: 1

ggplot(power_by_n, aes(x = beta, y = power, color = n)) +
  geom_line(linewidth = 1.1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "grey50") +
  scale_color_manual(values = c("20"="#cc5500", "50"="#2255aa", "200"="#2a7a2a"),
                    name = "n") +
  scale_y_continuous(labels = scales::percent, breaks = seq(0, 1, 0.2)) +
  labs(title = "Larger samples shift the entire power curve up",
       subtitle = "Same DGP, same alpha, different n -- only SE changes",
       x = "True beta*", y = "Power") +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"), legend.position = "top")

```

Larger samples shift the entire power curve up

Same DGP, same alpha, different n -- only SE changes



The OLS Estimator: Building Sampling Intuition

Goal: show concretely that $\hat{\beta}_1^{\text{OLS}}$ is a random variable whose distribution we can characterise.

5.1 Step 1 — Define the data-generating process (DGP)

```
# These are population-level parameters -- the researcher NEVER observes them.
beta0_true <- 1      # true intercept
beta1_true <- 3      # true slope (this is what we want to estimate)
sigma_u    <- 4      # standard deviation of the error term u
n_obs      <- 30     # observations per sample

cat("DGP: y_i = ", beta0_true, "+ ", beta1_true, "* x_i + u_i\n")

## DGP: y_i = 1 + 3 * x_i + u_i
cat("      u_i ~ N(0, ", sigma_u^2, ")\n")

##      u_i ~ N(0, 16 )
cat("      x_i ~ Uniform(0, 10)\n")

##      x_i ~ Uniform(0, 10)
cat("      n   =", n_obs, "observations per sample\n")

##      n   = 30 observations per sample
```

5.2 Step 2 — Draw one sample and inspect it

```
set.seed(7)

# Draw x from Uniform(0, 10) -- satisfies SLR.2 and SLR.3
x_sample <- runif(n_obs, min = 0, max = 10)

# Generate the error term: u ~ N(0, sigma_u^2)
u_sample <- rnorm(n_obs, mean = 0, sd = sigma_u)

# Apply the DGP to get y
y_sample <- beta0_true + beta1_true * x_sample + u_sample

# Look at the first 5 observations
df_sample <- data.frame(x = x_sample, u = u_sample, y = y_sample)
cat("First 5 observations:\n")

## First 5 observations:
print(round(head(df_sample, 5), 3))

##      x      u      y
## 1 9.889  1.871 32.538
## 2 3.977 -3.575  9.357
## 3 1.157 -1.229  3.242
## 4 0.697 -0.019  3.073
## 5 2.437  3.953 12.265
```

```
cat("\nNote: y = 1 + 3*x + u. For observation 1:\n")

##
## Note: y = 1 + 3*x + u. For observation 1:
cat("  y =", round(y_sample[1],3), "=",
    beta0_true, "+", beta1_true, "*", round(x_sample[1],3),
    "+", round(u_sample[1],3), "\n")

##  y = 32.538 = 1 + 3 * 9.889 + 1.871
```

5.3 Step 3 — Fit OLS and read off the estimate

```
# lm(y ~ x) minimises sum of (y_i - b0 - b1*x_i)^2 over b0 and b1.
# It returns the OLS estimates beta_hat_0 and beta_hat_1.
model      <- lm(y_sample ~ x_sample)
beta0_hat  <- coef(model)[1]   # estimated intercept
beta1_hat  <- coef(model)[2]   # estimated slope

cat("OLS results from this one sample:\n")

## OLS results from this one sample:
cat("  beta_hat_0 (intercept) =", round(beta0_hat, 3),
    "  [true value:", beta0_true, "]\n")

##  beta_hat_0 (intercept) = 1.017  [true value: 1 ]
cat("  beta_hat_1 (slope)      =", round(beta1_hat, 3),
    "  [true value:", beta1_true, "]\n")

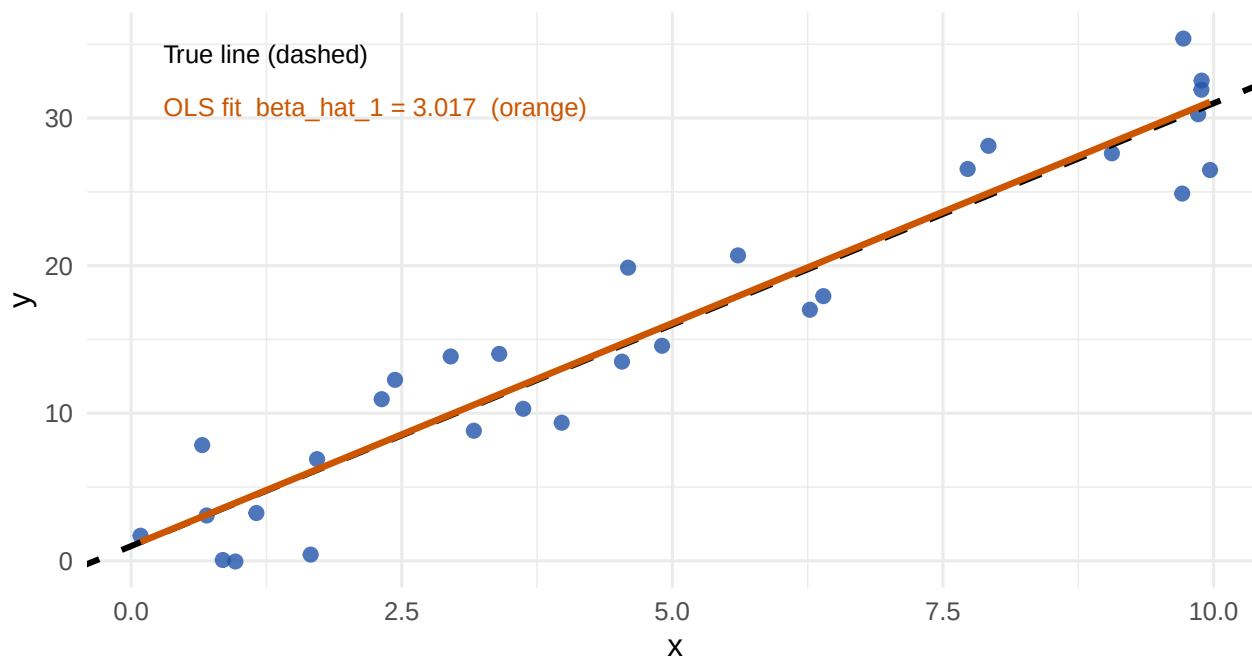
##  beta_hat_1 (slope)      = 3.017  [true value: 3 ]
cat("  Estimation error:      ", round(beta1_hat - beta1_true, 3), "\n")

##  Estimation error:      0.017

ggplot(df_sample, aes(x = x, y = y)) +
  geom_point(color = "#2255aa", size = 2, alpha = 0.8) +
  geom_abline(slope = beta1_true, intercept = beta0_true,
              color = "black", linewidth = 1, linetype = "dashed") +
  geom_smooth(method = "lm", se = FALSE, color = "#cc5500", linewidth = 1.1) +
  annotate("text", x = 0.3, y = max(y_sample) * 0.97,
          label = "True line (dashed)", hjust = 0, size = 3) +
  annotate("text", x = 0.3, y = max(y_sample) * 0.87,
          label = paste0("OLS fit  beta_hat_1 = ", round(beta1_hat, 3), " (orange)",
                          hjust = 0, size = 3, color = "#cc5500") +
  labs(title = "One sample: the OLS line versus the true population line",
       subtitle = paste0("The OLS estimate (", round(beta1_hat, 3),
                           ") differs from the truth (", beta1_true,
                           ") due to the random noise u."),
       x = "x", y = "y") +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"))
```


One sample: the OLS line versus the true population line

The OLS estimate (3.017) differs from the truth (3) due to the random noise u .



5.4 Step 4 — Repeat 40 times: the spaghetti plot

```
set.seed(42)
n_lines <- 40

# For each of 40 independent samples, run Steps 2-3 and save the coefficients.
# We only need the coefficients (b0, b1) -- not the raw data -- to draw the line.
all_coefs <- do.call(rbind, lapply(seq_len(n_lines), function(i) {
  x <- runif(n_obs, 0, 10)
  y <- beta0_true + beta1_true * x + rnorm(n_obs, 0, sigma_u)
  m <- lm(y ~ x)
  data.frame(sim = i, b0 = coef(m)[1], b1 = coef(m)["x"])
}))

cat("Summary of 40 OLS slope estimates:\n")

## Summary of 40 OLS slope estimates:
cat("  Mean:   ", round(mean(all_coefs$b1), 3),
    " (should be close to true beta1 =", beta1_true, ")\n")

##   Mean:    2.957  (should be close to true beta1 = 3 )
cat("  Min:    ", round(min(all_coefs$b1), 3), "\n")

##   Min:    2.542
cat("  Max:    ", round(max(all_coefs$b1), 3), "\n")

##   Max:    3.73
```

```

cat("  Range:  ", round(diff(range(all_coefs$b1)), 3),
    " -- this is sampling variability\n")

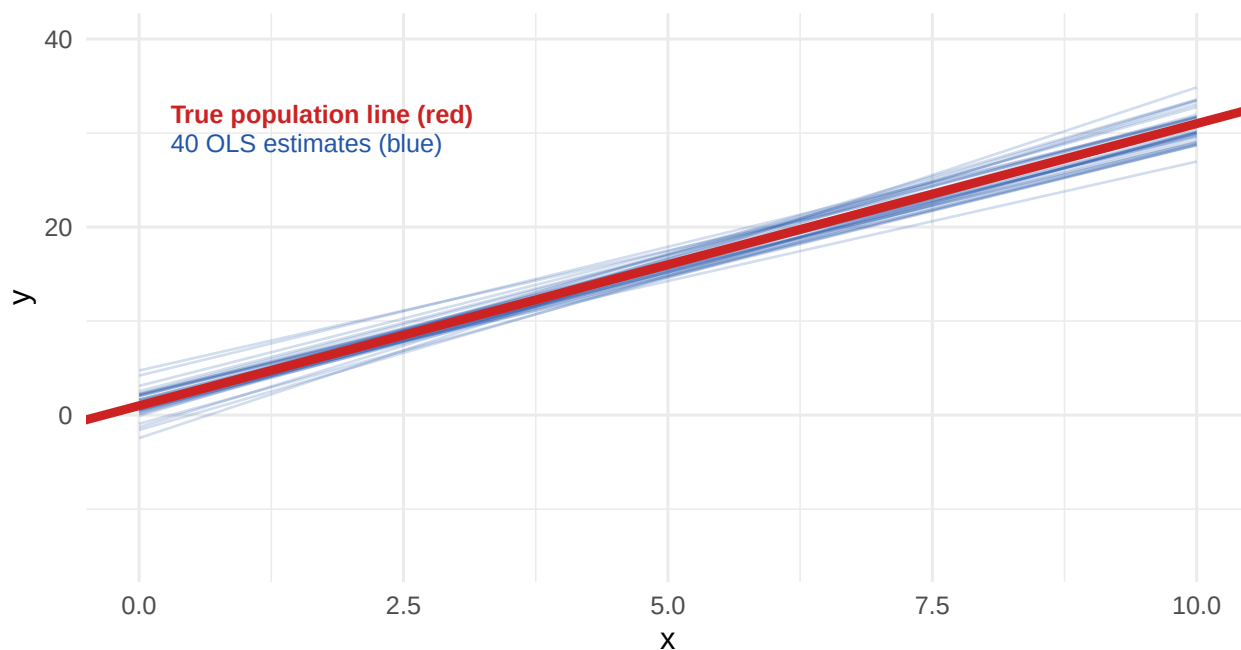
##  Range:  1.188 -- this is sampling variability
# Build (x, y) prediction pairs for each of the 40 lines
x_grid <- seq(0, 10, length.out = 80)
df_lines <- do.call(rbind, lapply(seq_len(nrow(all_coefs)), function(i) {
  data.frame(sim = i, x = x_grid,
             y = all_coefs$b0[i] + all_coefs$b1[i] * x_grid)
}))

ggplot() +
  geom_line(data = df_lines, aes(x = x, y = y, group = sim),
           color = "#2255aa", alpha = 0.2, linewidth = 0.45) +
  geom_abline(slope = beta1_true, intercept = beta0_true,
             color = "#cc2222", linewidth = 1.5) +
  annotate("text", x = 0.3, y = 32, color = "#cc2222", hjust = 0, size = 3,
         fontface = "bold", label = "True population line (red)") +
  annotate("text", x = 0.3, y = 29, color = "#2255aa", hjust = 0, size = 3,
         label = paste0(n_lines, " OLS estimates (blue)") +
  coord_cartesian(ylim = c(-15, 40)) +
  labs(title = "Each blue line is one OLS estimate from a different sample",
       subtitle = paste0("The cloud of lines centres on the true line -- this is unbiasedness: ",
                        "E[beta_hat_1] = beta_1"),
       x = "x", y = "y") +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"))

```

Each blue line is one OLS estimate from a different sample

The cloud of lines centres on the true line -- this is unbiasedness: $E[\beta_{\text{hat}_1}] = \beta_1$



5.5 Step 5 — Repeat 2,000 times: the sampling distribution

```

set.seed(1)
n_reps <- 2000

# Collect 2,000 slope estimates. replicate() runs the expression n_reps times
# and collects the result into a vector.
b1_all <- replicate(n_reps, {
  x <- runif(n_obs, 0, 10)
  y <- beta0_true + beta1_true * x + rnorm(n_obs, 0, sigma_u)
  coef(lm(y ~ x))["x"] # return only the slope
})

cat("Results from", n_reps, "OLS estimates:\n")

## Results from 2000 OLS estimates:

cat("  Mean:           ", round(mean(b1_all), 4),
    " (should be", beta1_true, ")\n")

##   Mean:           3.0037 (should be 3 )

cat("  Standard deviation: ", round(sd(b1_all), 4), "\n")

##   Standard deviation:  0.2576

# Theoretical SE from Wooldridge Theorem 2.2:
# SE(beta_hat_1) = sigma / sqrt(SST_x)
# For x ~ Uniform(0,10): Var(x) = 10^2/12, E[SST_x] = n * Var(x)
se_theoretical <- sigma_u / sqrt(n_obs * 10^2 / 12)
cat("  Theoretical SE:   ", round(se_theoretical, 4),
    " (from Theorem 2.2: sigma/sqrt(E[SST_x]))\n")

##   Theoretical SE:   0.253 (from Theorem 2.2: sigma/sqrt(E[SST_x]))

cat("  Agreement:        ",
    round(abs(sd(b1_all) - se_theoretical), 4), "difference\n")

##   Agreement:        0.0046 difference

# Plot the empirical histogram and overlay the theoretical normal density
ggplot(data.frame(b1 = b1_all), aes(x = b1)) +
  geom_histogram(aes(y = after_stat(density)), bins = 55,
    fill = "#2255aa", alpha = 0.7, color = "white") +
  # Theoretical density: N(beta1_true, se_theoretical^2)
  stat_function(fun = dnorm,
    args = list(mean = beta1_true, sd = se_theoretical),
    color = "#cc2222", linewidth = 1.1) +
  geom_vline(xintercept = beta1_true, linetype = "dashed") +
  annotate("text", x = beta1_true + 0.07, y = 1.20,
    label = paste0("True beta_1 = ", beta1_true), hjust = 0, size = 3) +
  annotate("text", x = beta1_true + 0.07, y = 1.00,
    label = paste0("Theoretical SE = ", round(se_theoretical, 3)),
    hjust = 0, size = 3, color = "#cc2222") +
  annotate("text", x = beta1_true + 0.07, y = 0.80,
    label = paste0("Empirical SD   = ", round(sd(b1_all), 3)),
    hjust = 0, size = 3, color = "#2255aa") +
  labs(title = paste0("Sampling distribution of beta_hat_1 (", n_reps, " replications)"),

```

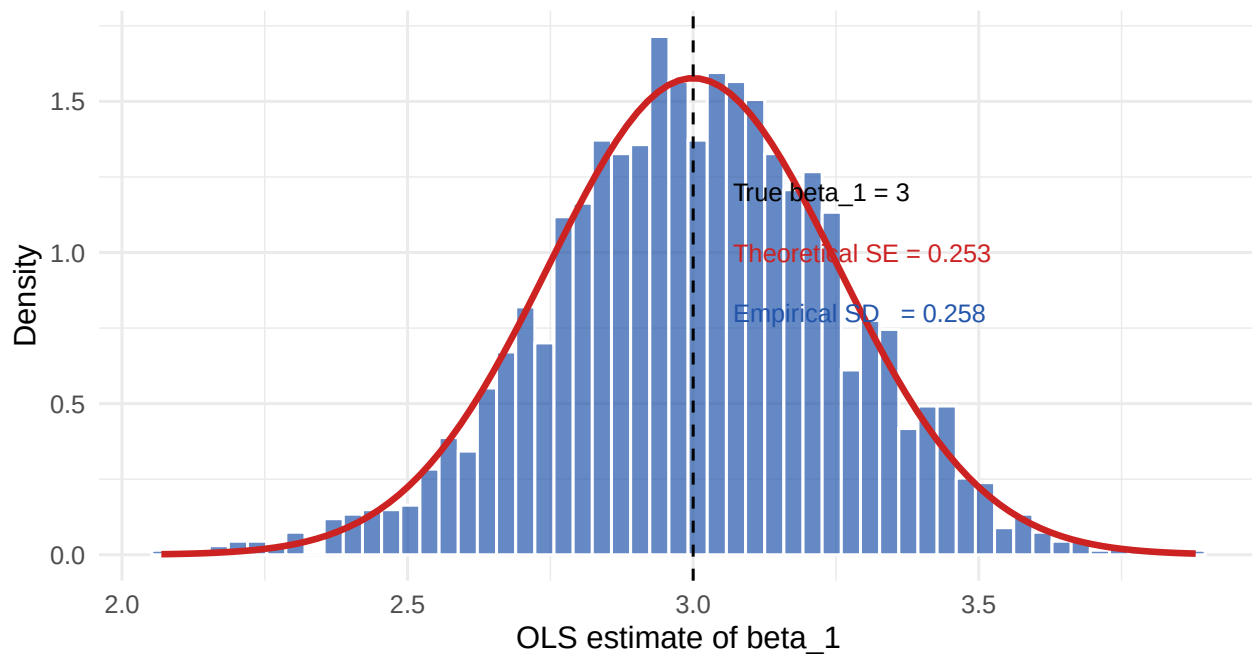
```

subtitle = paste0("Histogram = simulated; red curve = N(beta_1, SE^2) from Theorem 2.2. ",
                  "They match -- the theory is correct."),
x = "OLS estimate of beta_1", y = "Density" +
theme_minimal(base_size = 11) +
theme(plot.title = element_text(face = "bold"))

```

Sampling distribution of beta_hat_1 (2000 replications)

Histogram = simulated; red curve = $N(\beta_1, SE^2)$ from Theorem 2.2. They match



Confidence Intervals: What Does 95% Actually Mean?

Goal: construct 100 OLS confidence intervals and show concretely that approximately 95% of them capture the true $\beta_1 = 3$.

6.1 Step 1 — How does `confint()` build the interval?

```
# Fit OLS to one sample and inspect what confint() computes
set.seed(55)
x_ex <- runif(n_obs, 0, 10)
y_ex <- beta0_true + beta1_true * x_ex + rnorm(n_obs, 0, sigma_u)
m_ex <- lm(y_ex ~ x_ex)

# Extract ingredients manually
beta1_hat_ex <- coef(m_ex)["x_ex"]
se_hat_ex <- summary(m_ex)$coefficients["x_ex", "Std. Error"]
df_ex <- m_ex$df.residual # degrees of freedom = n - 2
t_crit_ex <- qt(0.975, df = df_ex) # t critical value (not z, because sigma unknown)

lower_manual <- beta1_hat_ex - t_crit_ex * se_hat_ex
upper_manual <- beta1_hat_ex + t_crit_ex * se_hat_ex

cat("Manual construction of the 95% CI:\n")

## Manual construction of the 95% CI:
cat("  beta_hat_1 =", round(beta1_hat_ex, 4), "\n")

##  beta_hat_1 = 2.8612
cat("  SE(beta_hat_1) =", round(se_hat_ex, 4), "\n")

##  SE(beta_hat_1) = 0.2616
cat("  t critical value (df =", df_ex, ") =", round(t_crit_ex, 4), "\n")

##  t critical value (df = 28 ) = 2.0484
cat("  Margin of error = t * SE =", round(t_crit_ex * se_hat_ex, 4), "\n")

##  Margin of error = t * SE = 0.5359
cat("  CI = [", round(lower_manual, 4), ",", round(upper_manual, 4), "]\n\n")

##  CI = [ 2.3254 , 3.3971 ]

# Now compare with confint()
cf_auto <- confint(m_ex, "x_ex", level = 0.95)
cat("confint() result:\n")

## confint() result:
cat("  CI = [", round(cf_auto[1], 4), ",", round(cf_auto[2], 4), "]\n")

##  CI = [ 2.3254 , 3.3971 ]
cat("They match -- confint() is doing exactly beta_hat +/- t * SE.\n")

## They match -- confint() is doing exactly beta_hat +/- t * SE.
```

6.2 Step 2 — Check if this specific CI captures the true value

```
# A CI [lower, upper] captures beta1_true if lower <= beta1_true <= upper.
captures_ex <- lower_manual <= beta1_true & beta1_true <= upper_manual

cat("True beta_1 =", beta1_true, "\n")

## True beta_1 = 3
cat("CI = [", round(lower_manual,3), ",", round(upper_manual,3), "]\n")

## CI = [ 2.325 , 3.397 ]
cat("Does this CI capture the true value?", captures_ex, "\n")

## Does this CI capture the true value? TRUE
if (captures_ex) {
  cat("-> YES. The interval contains the truth.\n")
} else {
  cat("-> NO. This is one of the ~5% that miss the truth.\n")
}

## -> YES. The interval contains the truth.
```

6.3 Step 3 — Repeat for 100 independent samples

```
set.seed(123)
n_ci <- 100 # number of CIs to construct

# For each of 100 samples: draw data, fit OLS, extract CI, check coverage.
ci_results <- lapply(seq_len(n_ci), function(i) {
  x <- runif(n_obs, 0, 10)
  y <- beta0_true + beta1_true * x + rnorm(n_obs, 0, sigma_u)
  m <- lm(y ~ x)
  cf <- confint(m, "x", level = 0.95)
  data.frame(
    sample = i,
    est = coef(m)["x"], # point estimate
    lower = cf[1],
    upper = cf[2],
    captures = cf[1] <= beta1_true & beta1_true <= cf[2]
  )
})

df_ci <- do.call(rbind, ci_results)

# Summary
cat("Results across", n_ci, "samples:\n")

## Results across 100 samples:
cat(" CIs that capture beta_1 = 3: ", sum(df_ci$captures), "\n")

## CIs that capture beta_1 = 3: 95
cat(" CIs that miss beta_1 = 3: ", sum(!df_ci$captures), "\n")
```

```
## CIs that miss beta_1 = 3: 5
cat(" Empirical coverage rate:      ", round(mean(df_ci$captures)*100, 1), "%\n")

## Empirical coverage rate:      95 %
cat(" Nominal coverage rate:      95%\n")

## Nominal coverage rate:      95%
```

6.4 Step 4 — Visualise all 100 intervals

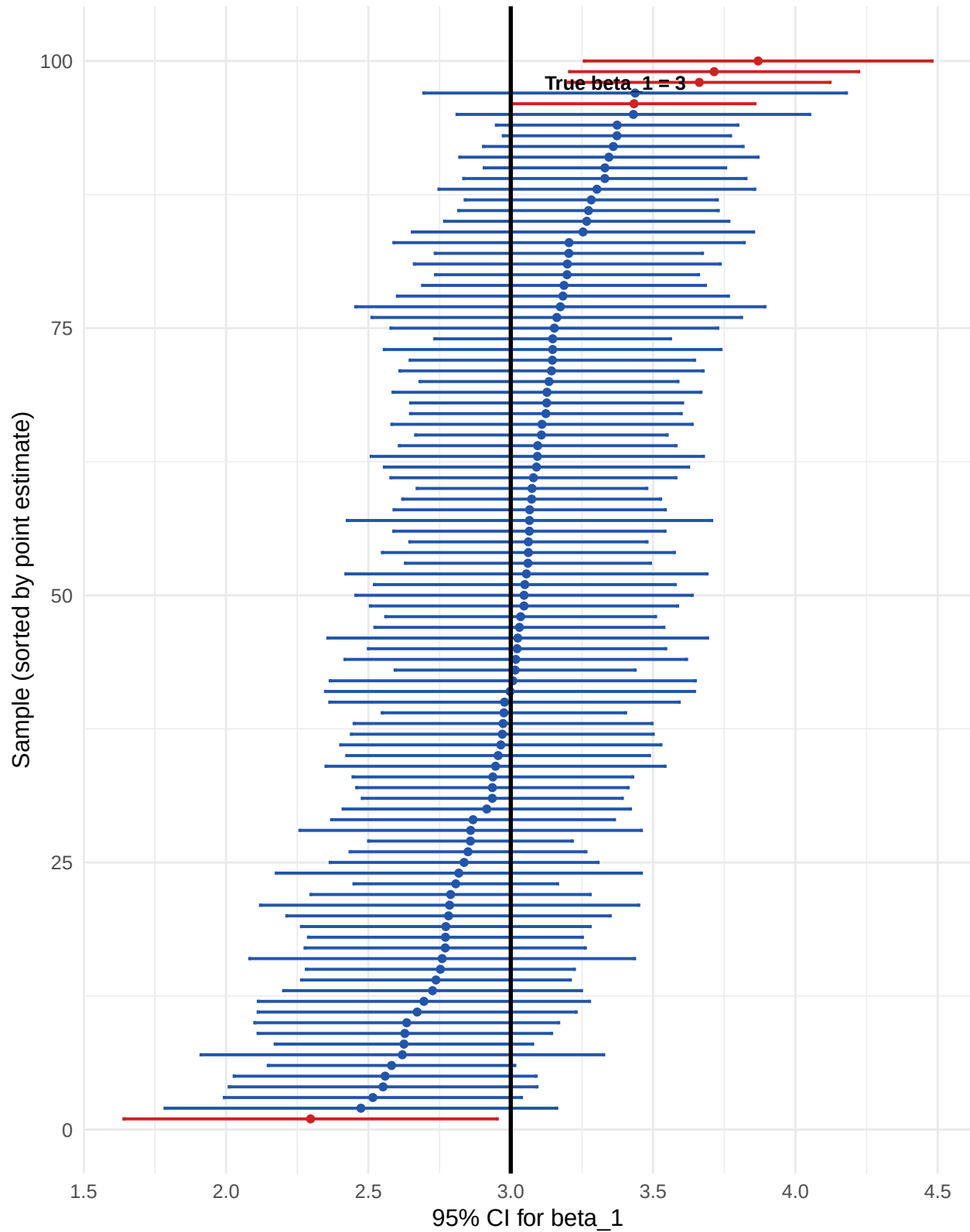
```
# Sort by point estimate to make the plot readable
df_ci <- df_ci[order(df_ci$est), ]
df_ci$rank <- seq_len(nrow(df_ci))

ggplot(df_ci, aes(x = est, xmin = lower, xmax = upper, y = rank,
                  color = captures)) +
  geom_errorbarh(height = 0.3, linewidth = 0.65) +
  geom_point(size = 1.3) +
  geom_vline(xintercept = beta1_true, linewidth = 0.9) +
  scale_color_manual(
    values = c("TRUE" = "#2255aa", "FALSE" = "#cc2222"),
    labels = c("TRUE" = paste0("Captures true beta_1 = ", beta1_true),
              "FALSE" = paste0("Misses true beta_1 = ", beta1_true)),
    name = NULL) +
  annotate("text", x = beta1_true + 0.12, y = 98,
          label = paste0("True beta_1 = ", beta1_true),
          hjust = 0, size = 3, fontface = "bold") +
  labs(
    title = paste0("100 repeated 95% CIs for beta_hat_1 (true beta_1 = ", beta1_true, ")"),
    subtitle = paste0(sum(df_ci$captures), " intervals (blue) capture the truth | ",
                     sum(!df_ci$captures), " intervals (red) miss it | ",
                     "Empirical coverage = ", round(mean(df_ci$captures)*100,1), "%"),
    x = "95% CI for beta_1",
    y = "Sample (sorted by point estimate)"
  ) +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"), legend.position = "top")
```

100 repeated 95% CIs for β_{hat_1} (true $\beta_1 = 3$)

95 intervals (blue) capture the truth | 5 intervals (red) miss it | Empirical coverage =

— Misses true $\beta_1 = 3$ — Captures true $\beta_1 = 3$



Reading this plot

Each horizontal bar is one confidence interval from one sample. The vertical black line is the true $\beta_1 = 3$. Blue bars cross the true line — they capture the truth. Red bars miss it entirely.

Notice that the intervals vary in both **position** (because $\hat{\beta}_1$ varies across samples) and **width** (because $SE(\hat{\beta}_1) = \hat{\sigma}/\sqrt{SST_x}$ also varies). About 5 out of 100 miss — consistent with 95% confidence.

The 95% is a property of the *method* (the factory), not of any single interval (the ring). Once a specific interval is computed, it either contains β_1 or it does not. There is no probability left for that specific interval.

6.5 Step 5 — Confirm coverage across different confidence levels

```
set.seed(999)
conf_levels <- c(0.80, 0.90, 0.95, 0.99)

# For each confidence level, repeat 2,000 times and check coverage
coverage_table <- do.call(rbind, lapply(conf_levels, function(lev) {
  # replicate() runs the expression 2,000 times and returns a vector
  cvg <- replicate(2000, {
    x <- runif(n_obs, 0, 10)
    y <- beta0_true + beta1_true * x + rnorm(n_obs, 0, sigma_u)
    cf <- confint(lm(y ~ x), "x", level = lev)
    cf[1] <= beta1_true & beta1_true <= cf[2]
  })
  # Also record the average width of the CI at this level
  wid <- replicate(500, {
    x <- runif(n_obs, 0, 10)
    y <- beta0_true + beta1_true * x + rnorm(n_obs, 0, sigma_u)
    cf <- confint(lm(y ~ x), "x", level = lev)
    cf[2] - cf[1]
  })
  data.frame(level = lev, label = paste0(lev*100, "%"),
             coverage = mean(cvg), avg_width = mean(wid))
}))

# Print the table
cat("Nominal vs. Empirical Coverage:\n\n")

## Nominal vs. Empirical Coverage:
cat(sprintf("%-12s %-18s %-14s\n", "Level", "Empirical coverage", "Avg CI width"))

## Level          Empirical coverage Avg CI width
cat(strrep("-", 46), "\n")

## -----
for (i in seq_len(nrow(coverage_table))) {
  cat(sprintf("%-12s %-18s %-14s\n",
             coverage_table$label[i],
             paste0(round(coverage_table$coverage[i]*100,1), "%"),
```

```

    round(coverage_table$avg_width[i], 3)))
}

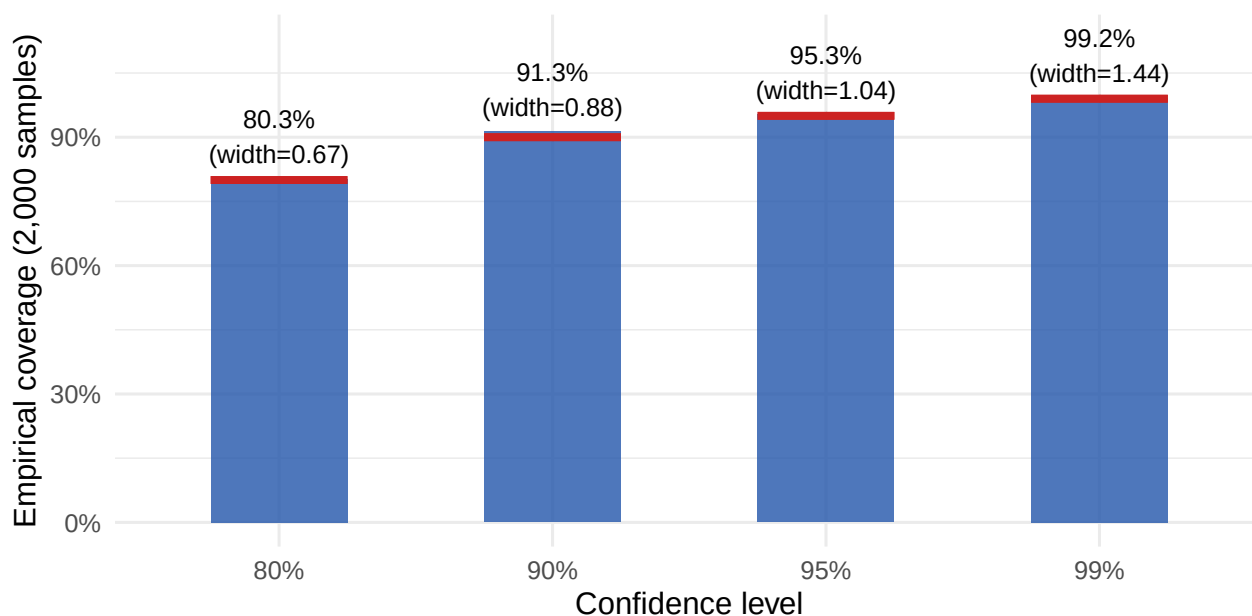
## 80%      80.3%      0.673
## 90%      91.3%      0.882
## 95%      95.3%      1.038
## 99%      99.2%      1.444

# Plot
ggplot(coverage_table, aes(x = label, y = coverage)) +
  geom_col(fill = "#2255aa", alpha = 0.8, width = 0.5) +
  geom_errorbar(aes(ymin = level, ymax = level), color = "#cc2222",
    linewidth = 1.3, width = 0.5) +
  geom_text(aes(label = paste0(round(coverage*100,1), "%\n(width=",
    round(avg_width,2), ")")),
    vjust = -0.3, size = 3) +
  scale_y_continuous(limits = c(0, 1.13), labels = scales::percent) +
  labs(title = "Higher confidence level = wider intervals = better coverage",
    subtitle = "Red tick = target coverage. Empirical (blue bars) tracks the target.",
    x = "Confidence level", y = "Empirical coverage (2,000 samples)") +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"))

```

Higher confidence level = wider intervals = better coverage

Red tick = target coverage. Empirical (blue bars) tracks the target.



Quick Reference: Every Function Used

```
# PROBABILITY FUNCTIONS
pnorm(x, mean, sd)      # P(X <= x) for X ~ N(mean, sd^2)
qnorm(p, mean, sd)      # value x such that P(X <= x) = p
dnorm(x, mean, sd)      # density of N(mean, sd^2) at x
pt(x, df)               # P(T <= x) for T ~ t_df
qt(p, df)               # value x such that P(T <= x) = p

# RANDOM NUMBER GENERATION
set.seed(n)              # fix the random seed for reproducibility
rnorm(n, mean, sd)       # draw n values from N(mean, sd^2)
runif(n, min, max)       # draw n values from Uniform(min, max)
replicate(n, expr)       # run expr n times, collect results into a vector

# DATA MANIPULATION
do.call(rbind, list)     # stack a list of data frames row-by-row
lapply(X, FUN)           # apply FUN to each element of X, return a list
sapply(X, FUN)           # same but try to simplify to a vector

# OLS
lm(y ~ x)               # fit OLS: minimise sum of squared residuals
coef(model)             # extract estimated coefficients (b0, b1, ...)
summary(model)          # full output: SE, t-values, p-values, R^2
confint(model, "x", 0.95) # 95% CI for the coefficient on x
                        # uses: beta_hat +/- t_{alpha/2, n-2} * SE(beta_hat)

# VECTORS OF LOGICALS
abs(Z) > z_crit          # TRUE where |Z| > 1.96 (rejection rule)
mean(logical_vector)     # proportion of TRUEs = empirical probability
sum(logical_vector)      # number of TRUEs
```

Three ideas that connect everything in this tutorial

- 1. Randomness lives in the data, not in the parameters.** The true $\beta_1 = 3$ is fixed. What changes across samples is $\hat{\beta}_1$ and $SE(\hat{\beta}_1)$ — so the confidence interval (built from these) is what is random, not β_1 .
- 2. Size is a calibration guarantee, not an accuracy guarantee.** $\alpha = 0.05$ means the testing procedure has a 5% false-alarm rate over many hypothetical experiments where H_0 is true. It says nothing about whether H_0 is true in your specific experiment.
- 3. Power depends on the signal-to-noise ratio $\delta = |\beta^*|/\sigma_{\hat{\beta}}$.** δ grows when the true effect is large or when we have more/better data (smaller SE). Both routes lead to higher power. This is why large datasets are needed to detect small economic effects.